

Z-TransformScript: Chapter 9 (z -transform, transfer functions of discrete-time systems)**with solutions****Learning goals**

- Compute z -transforms with their region of convergence (ROC) and invert rational $X(z)$ by partial fractions for causal, anticausal and two-sided sequences.
- Derive $H(z)$, poles and zeros of the RPM smoothers of Sheet 3 and decide stability from the pole positions.
- Read the magnitude response from the pole-zero map (geometric interpretation) and design a two-pole resonator for the knock frequency.
- Build a pole-zero explorer (polynomial roots, ASCII pole-zero map, frequency and impulse response).
- Implement the resonator in Q15 fixed point on the Arduino Uno and observe coefficient quantisation, zero-input limit cycles and overflow oscillations for $r \rightarrow 1$.

Background

The (bilateral) z -transform of $x[n]$ is $X(z) = \sum_{n=-\infty}^{\infty} x[n]z^{-n}$, defined in a ring-shaped ROC $r_1 < |z| < r_2$. An LTI system with the difference equation $\sum_k a_k y[n-k] = \sum_k b_k x[n-k]$ has the transfer function

$$H(z) = \frac{b_0 + b_1 z^{-1} + \dots + b_M z^{-M}}{a_0 + a_1 z^{-1} + \dots + a_N z^{-N}} = \frac{b_0}{a_0} z^{N-M} \frac{\prod_k (z - z_k)}{\prod_k (z - p_k)}.$$

A causal system is stable iff all poles lie inside the unit circle (then the ROC $|z| > \max |p_k|$ contains the unit circle), and its frequency response is $H(e^{j\Omega})$, $\Omega = 2\pi f/f_s$. Geometrically, $|H(e^{j\Omega})| = |\frac{b_0}{a_0}| \prod_k |e^{j\Omega} - z_k| / \prod_k |e^{j\Omega} - p_k|$: a pole close to the unit circle produces a peak, a zero on it a notch.

The **two-pole resonator** with poles $p_{1,2} = re^{\pm j\theta}$, $\theta = 2\pi f_0/f_s$,

$$H(z) = \frac{b_0}{1 + a_1 z^{-1} + a_2 z^{-2}}, \quad a_1 = -2r \cos \theta, \quad a_2 = r^2, \quad (1)$$

is the simplest digital knock filter. Throughout this sheet $f_0 = 6.5 \text{ kHz}$ and $f_s = 40 \text{ kHz}$ ($\theta = 58.5^\circ$).

Exercise 9.1 Analytical z -transforms**PAPER**

- Compute the z -transform and the ROC of $\delta[n-k]$, $\varepsilon[n]$, $a^n \varepsilon[n]$, $-a^n \varepsilon[-n-1]$ and $r^n \cos(\theta n) \varepsilon[n]$. Why is the ROC indispensable?
- Find all sequences with $X(z) = \frac{1}{(1 - 0.5z^{-1})(1 - 0.8z^{-1})}$ by partial fractions (one for each possible ROC). Which one is causal, which one has a DTFT? Check $x[0]$ and $x[1]$ of the causal one with the difference equation.
- RPM smoothers of Sheet 3.** Derive $H(z)$, poles and zeros of
 - the exponential smoother $y[n] = a y[n-1] + (1-a)x[n]$,
 - the M -point moving average.

When are they stable? Give $|H|$ at $\Omega = 0$ and $\Omega = \pi$ for $a = 0.9$ and the notch frequencies of the moving average for $M = 8$ at $f_s = 40 \text{ kHz}$. What happens to the pole at $z = 1$ of $(1 - z^{-M})/(1 - z^{-1})$?

- d) **Resonator.** Show that (1) has the poles $re^{\pm j\theta}$. Derive the stability triangle in the (a_1, a_2) -plane ($|a_2| < 1$, $|a_1| < 1 + a_2$) and mark the resonator.
- e) Using the geometric interpretation, show that the -3 dB bandwidth is $\Delta\Omega \approx 2(1 - r)$, i.e. $B \approx (1 - r)f_s/\pi$, for r close to 1. Choose r for $B = 400$ Hz. Compare with the impulse-invariant image $r = e^{-T/\tau}$ of the analog knock resonator of Sheet 7 ($\tau = 0.8$ ms).
- f) Determine b_0 such that $|H(e^{j\theta})| = 1$ and evaluate it for $r = 0.97$.

Solution

a) $\delta[n - k] \rightarrow z^{-k}$ (all z , except $z = 0$ for $k > 0$ and $z = \infty$ for $k < 0$). $\varepsilon[n] \rightarrow \frac{1}{1-z^{-1}}$, $|z| > 1$. $a^n \varepsilon[n] \rightarrow \sum_{n \geq 0} (az^{-1})^n = \frac{1}{1-az^{-1}}$, $|z| > |a|$. $-a^n \varepsilon[-n - 1] \rightarrow -\sum_{m \geq 1} (a^{-1}z)^m = \frac{1}{1-az^{-1}}$, $|z| < |a|$ – the same algebraic expression: only the ROC tells which sequence is meant.

$r^n \cos(\theta n) \varepsilon[n] = \frac{r^n}{2} (e^{j\theta n} + e^{-j\theta n}) \varepsilon[n] \rightarrow \frac{1-r \cos \theta z^{-1}}{1-2r \cos \theta z^{-1} + r^2 z^{-2}}$, $|z| > r$.

b) $X(z) = \frac{A}{1-0.5z^{-1}} + \frac{B}{1-0.8z^{-1}}$ with $A = \frac{1}{1-0.8/0.5} = -\frac{5}{3}$, $B = \frac{1}{1-0.5/0.8} = \frac{8}{3}$.

- $|z| > 0.8$: $x[n] = (-\frac{5}{3} 0.5^n + \frac{8}{3} 0.8^n) \varepsilon[n]$ (causal, ROC contains the unit circle $\Rightarrow$ DTFT exists).
- $0.5 < |z| < 0.8$: $x[n] = -\frac{5}{3} 0.5^n \varepsilon[n] - \frac{8}{3} 0.8^n \varepsilon[-n - 1]$ (two-sided, no DTFT).
- $|z| < 0.5$: $x[n] = \frac{5}{3} 0.5^n \varepsilon[-n - 1] - \frac{8}{3} 0.8^n \varepsilon[-n - 1]$ (anticausal, no DTFT).

Check: $y[n] = 1.3y[n-1] - 0.4y[n-2] + \delta[n]$ gives $y[0] = 1$, $y[1] = 1.3$; the formula: $-\frac{5}{3} + \frac{8}{3} = 1$, $-\frac{5}{6} + \frac{32}{15} = 1.3$.

c) i. $H(z) = \frac{1-a}{1-az^{-1}} = \frac{(1-a)z}{z-a}$: zero at 0, pole at a ; stable iff $|a| < 1$. $|H(e^{j0})| = 1$, $|H(e^{j\pi})| = \frac{1-a}{1+a} = 0.0526$ (-25.6 dB for $a = 0.9$). ii. $H(z) = \frac{1}{M} \sum_{k=0}^{M-1} z^{-k} = \frac{1}{M} \frac{1-z^{-M}}{1-z^{-1}} = \frac{z^M - 1}{M z^{M-1}(z-1)}$. The roots of $z^M = 1$ are $e^{j2\pi k/M}$; the one at $z = 1$ cancels the pole at $z = 1$, so there remain $M - 1$ zeros on the unit circle and $M - 1$ poles at $z = 0$: an FIR filter, always stable. For $M = 8$: notches at $k f_s/8 = 5, 10, 15, 20$ kHz.

d) $z^2 + a_1 z + a_2 = z^2 - 2r \cos \theta z + r^2 = (z - re^{j\theta})(z - re^{-j\theta})$. Stability triangle: for complex poles $|p|^2 = a_2 < 1$; for real poles the polynomial $P(z) = z^2 + a_1 z + a_2$ must have both roots in $(-1, 1)$, i.e. $P(1) = 1 + a_1 + a_2 > 0$ and $P(-1) = 1 - a_1 + a_2 > 0 \Rightarrow |a_1| < 1 + a_2$, $|a_2| < 1$. The resonator with $r = 0.97$ ($a_1 = -1.0136$, $a_2 = 0.9409$) lies inside, close to the upper edge $a_2 = 1$.

e) For Ω near θ the distance to the far pole is almost constant, $|e^{j\Omega} - re^{-j\theta}| \approx 2 \sin \theta$; the distance to the near pole is $|e^{j\Omega} - re^{j\theta}| \approx \sqrt{(1-r)^2 + (\Omega - \theta)^2}$. It grows by $\sqrt{2}$ for $|\Omega - \theta| = 1 - r$: $\Delta\Omega = 2(1 - r)$, $B = \frac{\Delta\Omega}{2\pi} f_s = (1 - r)f_s/\pi$. $B = 400$ Hz: $r = 1 - \pi B/f_s = 0.9686$. Impulse invariance: $r = e^{-T/\tau} = e^{-0.03125} = 0.9692$, $B = 392$ Hz, close to the analog 398 Hz – both views agree, because $1 - e^{-T/\tau} \approx T/\tau$ and $B_{\text{analog}} = 1/(\pi\tau)$.

f) $|H(e^{j\theta})| = \frac{b_0}{|1-re^{j\theta}e^{-j\theta}| |1-re^{-j\theta}e^{-j\theta}|} = \frac{b_0}{(1-r)|1-re^{-2j\theta}|}$, hence $b_0 = (1-r)\sqrt{1-2r \cos 2\theta + r^2}$; $r = 0.97$: $b_0 = 0.050393$. Peak gain without normalisation ($b_0 = 1$): $1/0.0504 = 19.8$ (26 dB) – important for the fixed-point scaling in Exercise 9.3.

Exercise 9.2 Pole-zero explorer

JSLINUX

Program code/jslinux/sheet09/pz_explorer.c reads the coefficients from the command line,

```
1 ./pz_explorer -fs 40000 -b 0.1 -a 1 -0.9 # exponential smoother
2 ./pz_explorer -fs 40000 -res 6500 0.97 # resonator, gain 1 at f0
```

computes poles and zeros, prints them in polar form (radius, angle in Hz), decides stability, draws the pole-zero map with `ap_pz()` and plots magnitude (dB) and phase over f and the impulse response.

- a) Poles and zeros are the roots of $z^L B(z)$ and $z^L A(z)$, $L = \max(M, N)$ (`tf_roots()` is given). Implement the Durand–Kerner iteration `poly_roots()` for a polynomial of degree n with $c_0 =$

1:

$$r_i \leftarrow r_i - \frac{c(r_i)}{\prod_{j \neq i} (r_i - r_j)}, \quad r_i^{(0)} = (0.4 + 0.9j)^{i+1}.$$

Use the small complex struct of the template (no `complex.h`).

- b) Implement the geometric evaluation `freq_geom()` and the impulse response from the difference equation. The program compares the geometric magnitude with the direct evaluation of B/A .
- c) Explore and report (pole-zero map, peak/notch, bandwidth, $|h[n]|$):
- exponential smoother $a = 0.9$; moving average $M = 8$;
 - resonator $r = 0.9, 0.97, 0.99$: compare the -3 dB bandwidth with $(1 - r)f_s/\pi$;
 - resonator $r = 1$ and $r = 1.02$: what does the program say, what does the impulse response do?
 - a notch filter at 6.5 kHz with zeros at $e^{\pm j\theta}$ and poles at $0.95e^{\pm j\theta}$. Give the coefficients and the notch width; why is $|H| \approx 1$ far from the notch?
- d) Why may the magnitude plot of an unstable system be misleading?

Solution

a)/b) Key parts of code/solutions/jslinux/sheet09/pz_explorer.c:

```

1 static int poly_roots(const double *p, int n, cpx *r)
2 {
3     cpx c[MAXC], w = cx(0.4, 0.9), num, den, d;
4     int i, j, it;
5     double change;
6     if (n < 1) return 0;
7     for (i = 0; i <= n; i++) c[i] = cx(p[i] / p[0], 0.0); /* monic */
8     r[0] = cx(1.0, 0.0);
9     for (i = 0; i < n; i++) r[i] = i ? cmul(r[i - 1], w) : cx(1.0, 0.0);
10    for (i = 0; i < n; i++) r[i] = cmul(r[i], w); /* (0.4+0.9j)^(i+1) */
11    for (it = 1; it <= 2000; it++) {
12        change = 0.0;
13        for (i = 0; i < n; i++) {
14            num = c[0]; /* Horner */
15            for (j = 1; j <= n; j++) num = cadd(cmul(num, r[j]), c[j]);
16            den = cx(1.0, 0.0);
17            for (j = 0; j < n; j++)
18                if (j != i) den = cmul(den, csub(r[i], r[j]));
19            d = cdiv(num, den);
20            r[i] = csub(r[i], d);
21            if (cabs_(d) > change) change = cabs_(d);
22        }
23        if (change < 1e-14) return it;
24    }
25    return it;
26 }
27
28 /* roots of z^L * (q0 + q1 z^-1 + ... + qK z^-K): exact zeros at z = 0 for
29    vanishing trailing coefficients, Durand-Kerner for the rest.
30    Leading zero coefficients (q0 = 0, ...) are roots "at infinity" and are
31    skipped. Returns the number of finite roots. */
32 static int tf_roots(const double *q, int K, int L, cpx *r)
33 {
34     double p[MAXC];
35     int i, lead = 0, n, nz = 0;
36     for (i = 0; i <= L; i++) p[i] = i <= K ? q[i] : 0.0;
37     while (lead <= L && p[lead] == 0.0) lead++; /* roots at infinity */
38     if (lead > L) return 0;
39     n = L - lead; /* degree */
40     while (n > 0 && p[lead + n] == 0.0) { r[nz++] = cx(0.0, 0.0); n--; }

```

```

41     poly_roots(p + lead, n, r + nz);
42     return nz + n;
43 }

```

```

1 static double freq_geom(double g, const cpx *z, int nz, const cpx *p, int np, double W)
2 {
3     cpx e = cx(cos(W), sin(W));
4     double m = fabs(g);
5     int k;
6     for (k = 0; k < nz; k++) m *= cabs(csub(e, z[k]));
7     for (k = 0; k < np; k++) m /= cabs(csub(e, p[k]));
8     return m;
9 }

```

The geometric and the direct magnitude agree to 10^{-14} dB (notch: 10^{-8} dB, relative error near the zero). Durand–Kerner converges quadratically for simple roots; for multiple roots (e.g. the $M - 1$ poles at $z = 0$ of the moving average, here found exactly by `tf_roots`) it is only linear.

c) Program results ($f_s = 40$ kHz, grid 10 Hz):

- Smoother $a = 0.9$: pole 0.9, zero 0; $|H|$ 0 dB at DC, -25.58 dB at $f_s/2$; $h[n] = 0.1 \cdot 0.9^n$ ($|h[59]| = 2.0 \cdot 10^{-4}$). Moving average $M = 8$: zeros at $\pm 5, \pm 10, \pm 15, 20$ kHz on the unit circle, 7 poles at 0; $h[n] = 1/8$ for $n = 0..7$, then 0.
- Resonator (b_0 normalised, poles at exactly 6500.0 Hz):

r	B measured	$(1 - r)f_s/\pi$	$ H $ at DC	$ H $ at $f_s/2$
0.90	1370 Hz	1273 Hz	-14.6 dB	-24.6 dB
0.97	400 Hz	382 Hz	-25.3 dB	-35.4 dB
0.99	140 Hz	127 Hz	-34.9 dB	-45.0 dB

The approximation is good for $r \rightarrow 1$ (the remaining difference for $r = 0.99$ is the 10 Hz grid). Smaller B means a longer impulse response: decay $\propto r^n$, time constant $-1/(f_s \ln r) = 0.82$ ms for $r = 0.97$ (cf. $\tau = 0.8$ ms of the knock burst).

- $r = 1$: “MARGINALLY STABLE (not BIBO stable)”, $h[n] = \sin((n + 1)\theta)$, a sinusoidal oscillator with constant amplitude; the magnitude plot shows an (on the 10 Hz grid finite) infinite peak. $r = 1.02$: “UNSTABLE”, $|h[n]|$ grows like 1.02^n ($|h[71]| = 0.157$ starting from 0.034).
- Notch: $b = \{1, -2 \cos \theta, 1\} = \{1, -1.044997, 1\}$, $a = \{1, -1.9 \cos \theta, 0.9025\} = \{1, -0.992748, 0.9025\}$. Zeros $0.5225 \pm 0.8526j$ ($|z| = 1$), poles $0.4964 \pm 0.8100j$ ($|p| = 0.95$), $|H| = -116$ dB at 6.5 kHz, $+0.42$ dB at DC, $+0.44$ dB at $f_s/2$; -3 dB stop band 6200–6800 Hz, width 600 Hz $\approx (1 - 0.95)f_s/\pi = 637$ Hz. Far from θ each zero and its neighbouring pole have almost the same distance to $e^{j\Omega}$, the ratios cancel ($|H| \approx 1$).

d) For poles outside the unit circle the ROC of the *causal* system does not contain the unit circle; $H(e^{j\Omega})$ is then a formal evaluation of the rational function, not the response to a sinusoid (which grows without bound). The program prints a warning in this case.

Exercise 9.3 Digital resonator in float and Q15

WOKWI JSLINUX

The resonator (1) is implemented twice in the header `resonator.h`: in float and in fixed point with signals in Q15 (`int16_t`, 1 LSB = 2^{-15}), coefficients in Q14 and an `int32_t` accumulator. The *same file* is used on JSLinux and on the Uno; you find it in both folders `code/jslinux/sheet09/` and `code/wokwi/sheet09_resonator/`.

- a) (**paper**) Why can a_1 not be stored in Q15 here? In which format is the accumulator, and how many bits does it need at most? Which quantisation step does $a_2 = r^2$ get, and how large is

the resulting change of the pole radius for $r = 0.9999$?

- b) Implement the functions `res_design`, `res_f_step` and `res_q_step` (rounding or truncation, saturation or wrap-around, and count the overflows). Note: `int` has 16 bits on the AVR – cast to `int32_t` before multiplying. Run the program `limitcycle.c` and compare quantised and exact pole positions (part (a) of the program) and the impulse responses (b).
- c) **Zero-input limit cycles.** Starting from $y[-1] = 0.5$ with $x = 0$, the float resonator decays to zero. Part (c) of the program measures the amplitude of the Q15 output after 5 s. For rounding, a limit cycle can persist as long as rounding maps $a_2 y$ back to y : derive the dead-band bound $|y| \leq \frac{0.5}{1-a_2}$ LSB. Compare with the measured amplitudes for $r \rightarrow 1$, rounding and truncation, and explain the frequency of the limit cycle.
- d) **Overflow.** Without normalisation ($b_0 = 1$) the gain at f_0 is about 20 (Exercise 9.1 f). Part (d) feeds a knock burst of amplitude $A = 0.1$ and 0.5 into the resonator with $r = 0.99$. Compare wrap-around and saturation. What is an *overflow oscillation*?
- e) **Uno.** Open the Wokwi project (Uno, potentiometer = r from 0.90 to 1.00, switch SW1 = rounding/truncation, SW2 = saturation/wrap, button = next test T0/T1/T2, see the sketch header) with your `resonator.h`. Reproduce the results of c) and d) for $r \approx 0.97, 0.99, 0.999$. Are the integer results identical to JSLinux? Could you also expect identical float results?
- f) Draw conclusions for a fixed-point knock filter in an ECU (choice of r , rounding, overflow handling, scaling).

Hint: The Uno computes the test signals itself; its ADC (about 9 kHz maximum, Sheet 5) could not sample a 6.5 kHz knock signal anyway. The tests are not real-time – we study the arithmetic; real-time knock filtering follows on Sheet 10.

Solution

a) $a_1 = -2r \cos \theta = -1.0136$ for $r = 0.97$ and up to -1.045 for $r \rightarrow 1$: outside the Q15 range $[-1, 1)$. Q14 ($[-2, 2)$, step 2^{-14}) is used for all coefficients. Products Q14 $\times$ Q15 are Q29; the sum of three products is bounded by $(|b_0| + |a_1| + |a_2|) 2^{29} < 4 \cdot 2^{29} = 2^{31}$: an `int32_t` is just sufficient. Step of a_2 : $2^{-14} = 6.1 \cdot 10^{-5}$, $\Delta r \approx \Delta a_2 / (2r) \leq 3.1 \cdot 10^{-5}$ – comparable with $1 - r = 10^{-4}$ for $r = 0.9999$: the bandwidth (1.3 Hz) is then only known to about $\pm 30\%$. Program output (a):

r	b_0	a_1	a_2	Q14: b_0, a_1, a_2	r_q	f_q [Hz]
0.9	0.162086	-0.940497	0.810000	2656, -15409, 13271	0.899999	6500.0
0.97	0.050393	-1.013647	0.940900	826, -16608, 15416	0.970009	6499.9
0.99	0.016968	-1.034547	0.980100	278, -16950, 16058	0.990001	6500.0
0.999	0.001704	-1.043952	0.998001	28, -17104, 16351	0.998992	6500.0
0.9999	0.000171	-1.044892	0.999800	3, -17120, 16381	0.999908	6499.9

Note also the coarse b_0 for $r \rightarrow 1$ (3 LSB: gain error up to 17%).

b) Key part of `resonator.h`:

```
1 static int16_t res_q_step(res_q_t *s, int16_t x)
2 {
3     int32_t acc, y;
4     acc = (int32_t)s->b0 * x - (int32_t)s->a1 * s->y1 - (int32_t)s->a2 * s->y2; /* Q29 */
5     if (s->rnd) acc += (int32_t)1 << 13; /* round to nearest */
6     y = acc >> 14; /* arithmetic shift: floor */
7     if (y > 32767 || y < -32768) {
8         s->novf++;
9         if (s->sat) y = y > 0 ? 32767 : -32768;
10        else y = (int16_t)(uint16_t)(y & 0xFFFF); /* two's complement wrap */
11    }
12    s->y2 = s->y1;
13    s->y1 = (int16_t)y;
```

```

14     return (int16_t)y;
15 }

```

Impulse $0.5\delta[n]$, $r = 0.97$, rounding: the Q15 output follows the float output with a maximum deviation of 7.2 LSB (peak 837 LSB); after 400 samples the float output is 0.000 LSB, the Q15 output still -2 LSB.

c) Zero input after $y[-1] = 0.5$: a limit cycle needs $|y[n]|$ to stay constant although the exact recursion multiplies the amplitude by $r < 1$ per sample. With rounding the oscillation behaves as if a_2 were 1 as long as $|\text{round}(a_2 y) - a_2 y| \leq 0.5$ can compensate the decay $(1 - a_2)|y|$, i.e. $(1 - a_2)|y| \leq 0.5$ (Jackson's dead band). Measured after 200 000 samples:

r	$a_{2,q}$	bound [LSB]	round: amp. [LSB]	f [Hz]	trunc.: amp. [LSB]	f [Hz]
0.9	13271	2.6	2	6680	1	0 (DC)
0.97	15416	8.5	7	6660	0	–
0.99	16058	25.1	11	6660	3	5720
0.995	16221	50.3	11	6660	0	–
0.999	16351	248.2	11	6660	0	–
0.9999	16381	2730.7	98	6500	172	6500
1.0	16384	∞	19217	6500	19207	6500

All rounding limit cycles respect the bound; they grow with r but stay far below it for $r \geq 0.99$. Their frequency is not f_0 : the limit cycle is a periodic integer sequence, here of period 6 samples ($f_s/6 = 6667$ Hz; the zero-crossing count gives 6660–6680). Truncation (arithmetic shift = rounding towards $-\infty$) usually lets the output die out but can lock into a constant -1 LSB (DC limit cycle, $r = 0.9$) or into oscillations ($r = 0.99$: period 7, 5714 Hz; $r = 0.9999$). Example ($r = 0.99$, rounding): $\dots, -6, -11, -5, 6, 11, 5, -6, \dots$. For $r = 1$ the system is an oscillator (amplitude $0.5/\sin \theta \cdot 32768 \approx 19217$ LSB), not a limit cycle.

d) $r = 0.99$, $b_0 = 1$, burst $Ae^{-t/\tau} \sin(2\pi f_0 t)$ for 10 ms, then 0:

case	float peak	overflows	max $ y $ in the last 25 ms [LSB]
$A = 0.1$, wrap-around	1.08	39967	21740
$A = 0.1$, saturation	1.08	5	11
$A = 0.5$, wrap-around	5.42	24	11
$A = 0.5$, saturation	5.42	34	11

With two's complement wrap-around, an overflow turns a large positive value into a large negative one; for $|a_1| + |a_2| > 1$ this can sustain a full-scale oscillation without input (*overflow oscillation*): for $A = 0.1$ the filter overflows in practically every sample for the whole second. Whether it happens depends on the trajectory ($A = 0.5$ recovered by chance). Saturation limits the error to a clipping and the filter recovers to the small rounding limit cycle (11 LSB).

e) The integer path uses only `int16_t/int32_t` arithmetic with identical rounding, so the Uno must produce *bit-identical* Q15 results to JSLinux for the same r (the potentiometer gives $r = 0.9 + 0.1 \cdot \text{adc}/1023$, so use the printed r and Q14 coefficients for the comparison, e.g. $\text{adc} = 921 \rightarrow r = 0.99003$). The float path is single precision on both, but `sinf/expf` of `avr-libc` and the evaluation order may differ in the last bit; also a 32-bit x86 compiler (JSLinux) may keep intermediates in 80-bit registers. Small float differences are therefore possible, integer differences are not.

f) For $B \approx 400$ Hz ($r \approx 0.97$) the rounding limit cycle is 7 LSB (-73 dBFS), harmless compared with the sensor noise. Rules: use rounding, always saturate (never wrap), scale the input with the normalised b_0 (or reduce the input range) so that the output cannot exceed full scale, store coefficients in Q14 when $|a_1| > 1$, avoid $r \rightarrow 1$ (narrow filters need more bits: 32-bit state or error feedback), or use floating point on the ESP32 (Sheet 10).

Deliverables

Solutions of 9.1 (partial fractions with all ROCs, stability triangle, bandwidth formula and r for 400 Hz); the pole-zero explorer with the screenshots (ASCII plots) and the result table of 9.2 c); the tables of 9.3 a), c) and d) from JSLinux and the corresponding Uno outputs (T0–T2 for at least two values of r) with your conclusions 9.3 f).